

Tugas 10: Tugas Praktikum Mandiri 10 – Machine Learning

Yurida Yahsya 1 - 0110224100 1*

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: 0110224100@student.nurulfikri.ac.id – email mahasiswa 1

1. Tugas Mandiri 10

➤ **Latihan 1**

colab.research.google.com/drive/1POg4y6se-jlK706ul2mcyXMsui_7-QD4#scrollTo=R9u/SnYfBg3

Tugas_Mandiri10_Latihan1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[1] `import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score
import numpy as np`

[23] `data = {
 'Temperatur Udara (°C)': [10, 25, 15, 20, 18, 20, 22, 24],
 'Kecepatan Angin (km/jam)': [0, 0, 5, 3, 7, 10, 5, 6],
 'Persepsi': ['Dingin', 'Panas', 'Panas', 'Dingin', 'Panas', 'Dingin', 'Dingin', 'Panas']
}
df = pd.DataFrame(data)`

[24] `# Ubah Pandas DataFrame menjadi NumPy Array eksplisit
X_train = df[['Temperatur Udara (°C)', 'Kecepatan Angin (km/jam)']].values
y_train = df['Persepsi'].values
X_test = np.array([[16, 3]])`

[25] `print("Data Training:")
print(df)
print("\nData Test: [16, 3])`

Data Training:

	Temperatur Udara (°C)	Kecepatan Angin (km/jam)	Persepsi
0	10	0	Dingin
1	25	0	Panas
2	15	5	Dingin
3	20	3	Panas
4	18	7	Dingin
5	20	10	Dingin
6	22	5	Dingin
7	24	6	Panas

Data Test:

	Temperatur Udara (°C)	Kecepatan Angin (km/jam)	Persepsi
0	16	3	Panas

The image displays two screenshots of a Google Colab notebook titled "Tugas_Mandiri10_Latihan1.ipynb".

Top Screenshot: Shows the initial data and a prediction for a specific test point.

```
[25] print("Data Training:")
print(df)
print("\nData Test: [16, 3]")
```

Output for Data Training:

	Temperatur Udara (°C)	Kecepatan Angin (km/jam)	Persepsi
0	10	0	Dingin
1	25	0	Panas
2	15	5	Dingin
3	20	3	Panas
4	18	7	Dingin
5	20	10	Dingin
6	22	5	Panas
7	24	6	Panas

Data Test: [16, 3]

```
[34] # --- BAGIAN A: BAGAIMANA PERSEPSI MARRY SAAT TEMPERATUR UDARA 16C DAN KECEPATAN ANGIN3KM/JAM? ---
print(f"\n# A. Prediksi MARRY menggunakan K Terbaik (K={best_k})")
knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train, y_train)
prediksi_B = knn_best.predict(X_test)
print(f"Prediksi MARRY saat K terbaik (K={best_k}) adalah: {(prediksi_B[0])}")
```

Output: # A. Prediksi MARRY menggunakan K Terbaik (K=1) Prediksi MARRY saat K terbaik (K=1) adalah: "Dingin"

Bottom Screenshot: Shows the cross-validation process to find the optimal K value.

```
[28] # --- BAGIAN B: MENENTUKAN NILAI K YANG TEPAT BERDASARKAN AKURASI YANG PALING BAIK ---
print("\n# B. Menentukan Nilai K Terbaik")

# Coba nilai K
k_values = list(range(1, 4, 2)) # Mencoba K=1, 3

accuracies = []
for k in k_values:
    # Menggunakan Cross-Validation
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train, y_train, cv=2)
    accuracies.append(scores.mean())
print(f"Rata-rata Akurasi dengan K={k}: {scores.mean():.4f}")
```

Output: # B. Menentukan Nilai K Terbaik Rata-rata Akurasi dengan K=1: 0.8750 Rata-rata Akurasi dengan K=3: 0.7500

```
[29] # Cari K dengan akurasi rata-rata tertinggi
best_k_index = np.argmax(accuracies)
best_k = k_values[best_k_index]
best_accuracy = accuracies[best_k_index]

print(f"\nNilai K terbaik berdasarkan akurasi Cross-Validation adalah K={best_k} (Akurasi Rata-rata: {best_accuracy:.4f})")
```

Output: Nilai K terbaik berdasarkan akurasi Cross-Validation adalah K=1 (Akurasi Rata-rata: 0.8750)

1. Penjelasan langkah-langkah analisis

A. Persiapan Data

- **Data Training:** Dibuat DataFrame df dari data yang diberikan, yang berisi 7 baris data pelatihan (indeks 0 hingga 6). Fitur (X_train) adalah Temperatur Udara dan Kecepatan Angin, sedangkan target (y_train) adalah Persepsi.
- **Data Uji (Test Data):** Dibuat array X_test yang berisi satu titik data untuk diprediksi: Temperatur Udara 16C dan Kecepatan Angin 3 km/jam.

B. Penentuan Nilai K Terbaik

- Dilakukan iterasi untuk mencari nilai K (jumlah tetangga terdekat) yang paling optimal. Dalam kode, nilai K yang dicoba adalah K=1, 3 (karena range(1, 4, 2) menghasilkan 1 dan 3).
- Validasi Silang (*Cross-Validation*): Untuk setiap nilai K, dilakukan Validasi Silang menggunakan fungsi `cross_val_score`. Ini adalah teknik untuk mengukur seberapa baik model akan digeneralisasikan ke dataset independen. Rata-rata akurasi dari hasil validasi silang dicatat untuk setiap nilai K.
- Penentuan Terbaik: Nilai K dengan rata-rata akurasi validasi silang tertinggi dipilih sebagai `best_k`.

C. Prediksi Akhir

- Pelatihan Model (dengan K Terbaik): Model K-NN dilatih kembali menggunakan seluruh data training (`X_train`, `y_train`) dengan nilai `best_k` yang telah ditemukan.
- Prediksi: Model yang sudah dilatih digunakan untuk memprediksi Persepsi dari data uji (`X_test`).

2. Interpretasi hasil perhitungan dan prediksi

A. Hasil Perhitungan Nilai K Terbaik

- Interpretasi: Berdasarkan hasil validasi silang, nilai K=1 memberikan rata-rata akurasi tertinggi (0.8750 atau 87.5%). Ini berarti model K-NN dengan satu tetangga terdekat memiliki performa generalisasi terbaik untuk dataset ini.

- Nilai K Terbaik: K=1

B. Hasil Prediksi Akhir

- Data Uji: Temperatur Udara 16C, Kecepatan Angin 3 km/jam.
- Prediksi: "Dingin"
- Interpretasi: Setelah model K-NN dilatih dengan K=1, ia memprediksi bahwa kondisi cuaca dengan temperatur 16C dan kecepatan angin 3 km/jam akan dipersepsikan sebagai "Dingin".

Dalam K-NN dengan K=1, prediksi didasarkan pada label dari satu titik data pelatihan terdekat. Diperkirakan titik data [16, 3] memiliki jarak terpendek ke salah satu data pelatihan yang berlabel "Dingin".

3. Kesimpulan akhir dari setiap studi kasus

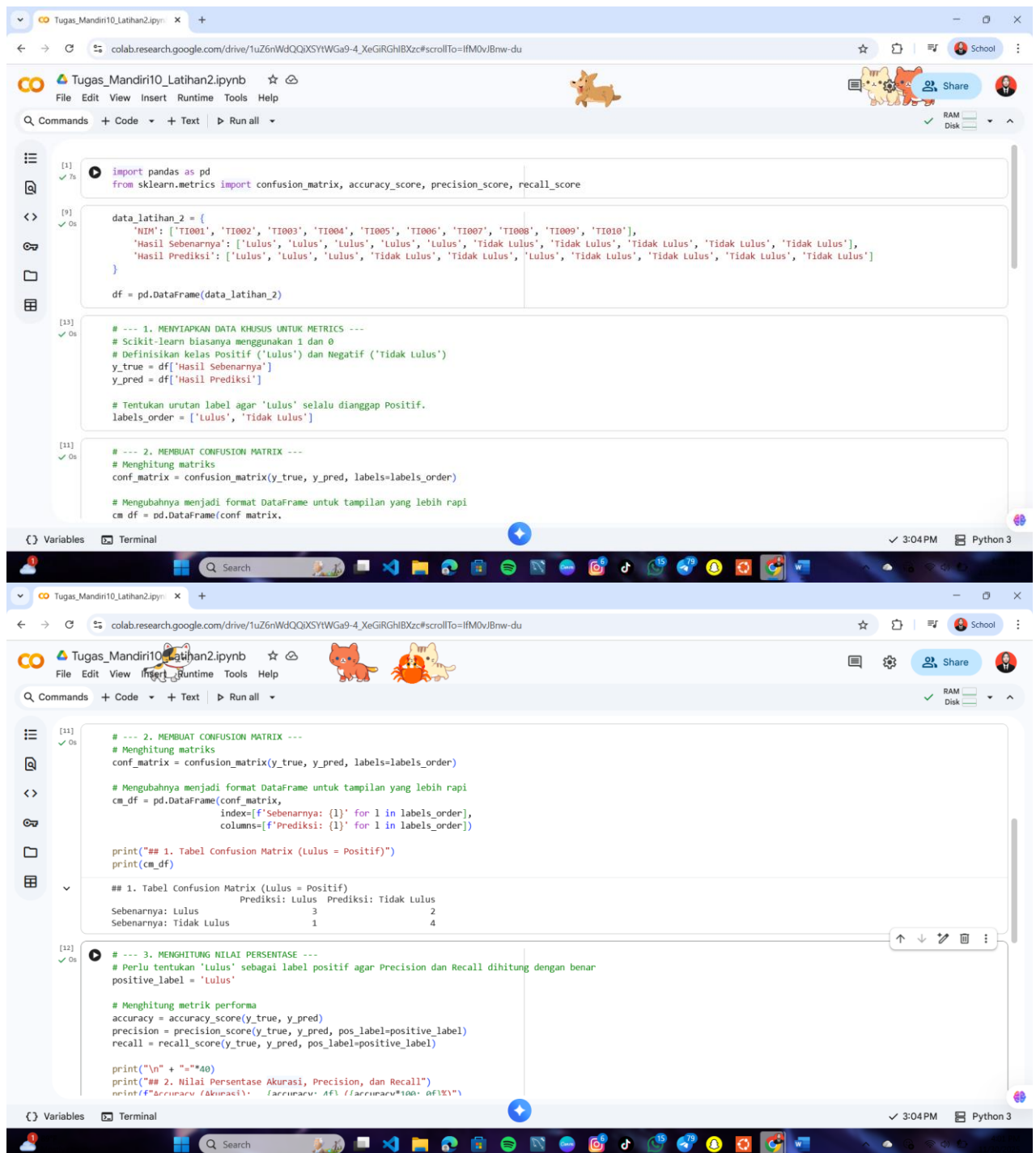
Studi kasus ini berhasil mengimplementasikan model klasifikasi K-Nearest Neighbors (K-NN) untuk memprediksi persepsi cuaca.

1. Optimasi Model: Melalui proses validasi silang, nilai K=1 ditemukan sebagai parameter model yang paling optimal, menghasilkan rata-rata akurasi sebesar 0.8750.

2. Prediksi Kasus: Dengan menggunakan model K-NN terbaik ($K=1$), kondisi cuaca dengan Temperatur Udara 16 C dan Kecepatan Angin 3 km/jam diprediksi akan memiliki Persepsi "Dingin".

Secara keseluruhan, algoritma K-NN efektif dalam mengklasifikasikan persepsi cuaca berdasarkan data historis yang tersedia.

➤ Latihan 2



```
[1] import pandas as pd
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

[9] data_latihan_2 = {
    'TIM': ['T1001', 'T1002', 'T1003', 'T1004', 'T1005', 'T1006', 'T1007', 'T1008', 'T1009', 'T1010'],
    'Hasil Sebenarnya': ['Lulus', 'Lulus', 'Lulus', 'Lulus', 'Lulus', 'Tidak Lulus', 'Tidak Lulus', 'Tidak Lulus', 'Tidak Lulus', 'Tidak Lulus'],
    'Hasil Prediksi': ['Lulus', 'Lulus', 'Lulus', 'Tidak Lulus', 'Tidak Lulus', 'Lulus', 'Tidak Lulus', 'Tidak Lulus', 'Tidak Lulus', 'Tidak Lulus']
}

df = pd.DataFrame(data_latihan_2)

[13] # --- 1. MENYIAPKAN DATA KHUSUS UNTUK METRICS ---
# Scikit-learn biasanya menggunakan 1 dan 0
# Definisikan kelas Positif ('Lulus') dan Negatif ('Tidak Lulus')
y_true = df['Hasil Sebenarnya']
y_pred = df['Hasil Prediksi']

# Tentukan urutan label agar 'Lulus' selalu dianggap Positif.
labels_order = ['Lulus', 'Tidak Lulus']

[11] # --- 2. MEMBUAT CONFUSION MATRIX ---
# Menghitung matriks
conf_matrix = confusion_matrix(y_true, y_pred, labels=labels_order)

# Mengubahnya menjadi format DataFrame untuk tampilan yang lebih rapi
cm_df = pd.DataFrame(conf_matrix,
    index=[f'Sebenarnya: {l}' for l in labels_order],
    columns=[f'Prediksi: {l}' for l in labels_order])

print("## 1. Tabel Confusion Matrix (Lulus = Positif)")
print(cm_df)

## 1. Tabel Confusion Matrix (Lulus = Positif)
                Prediksi: Lulus  Prediksi: Tidak Lulus
Sebenarnya: Lulus                3                 2
Sebenarnya: Tidak Lulus          1                 4

[12] # --- 3. MENGHITUNG NILAI PERSENTASE ---
# Perlu tentukan 'Lulus' sebagai label positif agar Precision dan Recall dihitung dengan benar
positive_label = 'Lulus'

# Menghitung metrik performa
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, pos_label=positive_label)
recall = recall_score(y_true, y_pred, pos_label=positive_label)

print("\n" + "="*40)
print("## 2. Nilai Persentase Akurasi, Precision, dan Recall")
print(f"Akurasi: {accuracy:.4f} / Akurasi: {accuracy*100:.4f} %")
```

The screenshot shows a Google Colab notebook titled 'Tugas_Mandiri10_Latihan2.ipynb'. The code cell [12] contains the following Python code:

```
## 2. MENGENAL KONSEP METRIK (AKURASI, PRECISION, RECALL)

# Contoh data hasil prediksi (y_pred) dan sebenarnya (y_true)
y_true = ['Lulus', 'Tidak Lulus', 'Lulus', 'Tidak Lulus']
y_pred = ['Lulus', 'Lulus', 'Tidak Lulus', 'Tidak Lulus']

# Membuat matriks kebingungan (Confusion Matrix)
cm = confusion_matrix(y_true, y_pred)

# Menampilkan matriks kebingungan
print("Matriks Kebingungan (Confusion Matrix):")
print(cm)

# Menghitung metrik performa
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, pos_label='Lulus')
recall = recall_score(y_true, y_pred, pos_label='Lulus')

print("\n" + "="*40)
print("## 2. Nilai Persentase Akurasi, Precision, dan Recall")
print(f"Accuracy (Akurasi): {accuracy:.4f} ({accuracy*100:.0f}%)")
print(f"Precision (Presisi): {precision:.4f} ({precision*100:.0f}%)")
print(f"Recall (Sensitivitas): {recall:.4f} ({recall*100:.0f}%)")
print("\n" + "="*40)
```

The output of the code shows the Confusion Matrix and the calculated performance metrics:

```
Matriks Kebingungan (Confusion Matrix):
[[ 3  2]
 [ 1  4]]

## 2. Nilai Persentase Akurasi, Precision, dan Recall
Accuracy (Akurasi):  0.7000 (70%)
Precision (Presisi):  0.7500 (75%)
Recall (Sensitivitas): 0.6000 (60%)
```

1. Penjelasan langkah-langkah analisis

A. Persiapan Data

- Data Input: Disediakan data hasil sebenarnya (y_{true}) dan hasil prediksi model (y_{pred}) untuk 10 sampel (ID T1001 - T1010). Kedua label ini berisi kategori 'Lulus' dan 'Tidak lulus'.
- Definisi Label Positif: Untuk menghitung metrik seperti Precision dan Recall dengan benar, label 'lulus' didefinisikan sebagai Label Positif ($positive_label$).

B. Pembuatan Confusion Matrix

- Fungsi: `confusion_matrix` dari `sklearn.metrics` digunakan untuk membuat matriks tabulasi yang membandingkan y_{true} dan y_{pred} .
- Struktur: Confusion Matrix adalah tabel yang menunjukkan jumlah hasil prediksi yang benar dan salah.
 - Baris mewakili Hasil Sebenarnya.
 - Kolom mewakili Hasil Prediksi.

C. Perhitungan Metrik Performa

Metrik-metrik berikut dihitung untuk mengukur kinerja model:

1. Accuracy (Akurasi): Dihitung menggunakan `accuracy_score`. Mengukur proporsi total prediksi yang benar dari keseluruhan data.
2. Precision (Presisi): Dihitung menggunakan `precision_score`. Mengukur proporsi hasil 'lulus' yang diprediksi yang benar-benar 'lulus' (mengatasi False Positive).
3. Recall (Sensitivitas): Dihitung menggunakan `recall_score`. Mengukur proporsi semua kasus 'lulus' yang benar-benar berhasil diprediksi 'lulus' (mengatasi False Negative).

2. Interpretasi hasil perhitungan dan prediksi

A. Tabel *Confusion Matrix*

- TP (True Positive): 3 Model memprediksi **Lulus**, dan hasilnya benar (Lulus).
- FN (False Negative): 1 Model memprediksi **Tidak lulus**, padahal hasilnya sebenarnya Lulus. Ini adalah Type II Error.
- FP (False Positive): 2 Model memprediksi **Lulus**, padahal hasilnya sebenarnya Tidak lulus. Ini adalah Type I Error.
- TN (True Negative): 4 Model memprediksi **Tidak lulus**, dan hasilnya benar (Tidak lulus).

B. Nilai Persentase Metrik Performa

- Akurasi (70%): Secara keseluruhan, 70% dari semua prediksi model adalah benar (3 Lulus + 4 Tidak Lulus dari total 10).
- Presisi (60%): Dari semua yang diprediksi Lulus, hanya 60% yang benar-benar Lulus. Nilai ini relatif rendah karena adanya 2 False Positive (model salah memprediksi 2 orang yang sebenarnya Tidak Lulus sebagai Lulus).
- Recall (75%): Dari semua yang sebenarnya Lulus, model berhasil mengidentifikasi 75% sebagai Lulus. Nilai ini cukup baik, namun adanya 1 False Negative menunjukkan model gagal mengidentifikasi satu kasus Lulus.

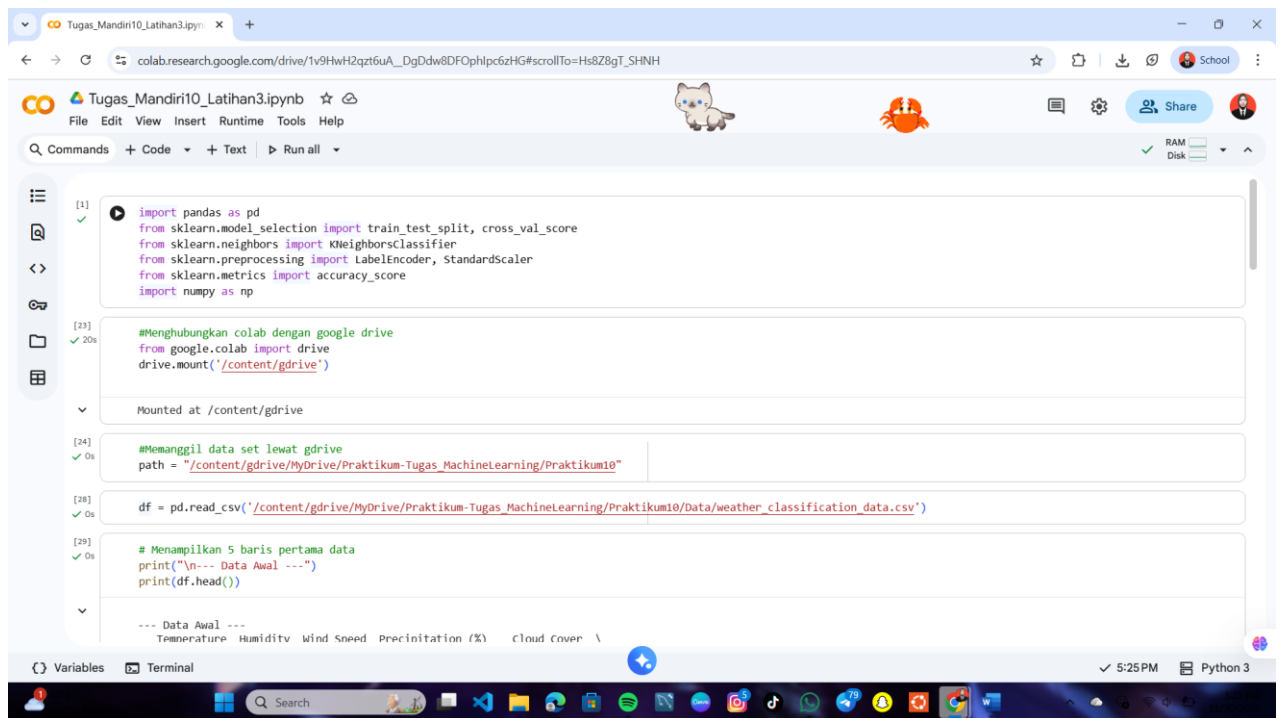
3. Kesimpulan akhir dari setiap studi kasus

Evaluasi model klasifikasi, dengan menetapkan 'Lulus' sebagai label positif, menunjukkan kinerja model yang cukup memadai namun perlu diwaspadai dari sisi kualitas prediksi.

- Akurasi Total (70%): Model berhasil membuat prediksi yang benar sebanyak 7 dari 10 kasus, menunjukkan kinerja umum yang wajar.
- Kelemahan pada Presisi (60%): Nilai Presisi yang rendah mengindikasikan bahwa model terlalu banyak membuat prediksi Lulus yang salah (False Positive). Dari semua yang diprediksi Lulus, hanya 60% yang benar-benar Lulus.
- Kekuatan pada Recall (75%): Model cukup baik dalam menangkap atau mengidentifikasi kasus positif (Lulus). 75% dari semua kasus yang seharusnya Lulus berhasil diprediksi dengan benar.

Model ini dapat diandalkan dalam menemukan sebagian besar kasus 'Lulus' (*high recall*). Namun, kelemahannya terletak pada rendahnya Presisi, yang berarti model sering memberikan alarm palsu ('Lulus' padahal 'Tidak Lulus'). Jika tujuan utama adalah memastikan bahwa setiap prediksi 'Lulus' adalah benar, model memerlukan optimasi lebih lanjut untuk mengurangi False Positive.

➤ Latihan 3



The screenshot shows a Google Colab notebook titled 'Tugas_Mandiri10_Latihan3.ipynb'. The code in the first cell imports necessary libraries: pandas, sklearn (train_test_split, cross_val_score, KNeighborsClassifier, LabelEncoder, StandardScaler, accuracy_score), and numpy. The second cell connects to Google Drive and mounts it at '/content/gdrive'. The third cell sets the path to the data file. The fourth cell reads the CSV file into a DataFrame. The fifth cell prints the first five rows of the data.

```
[1] import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score
import numpy as np

[23] #Menghubungkan colab dengan google drive
from google.colab import drive
drive.mount('/content/gdrive')

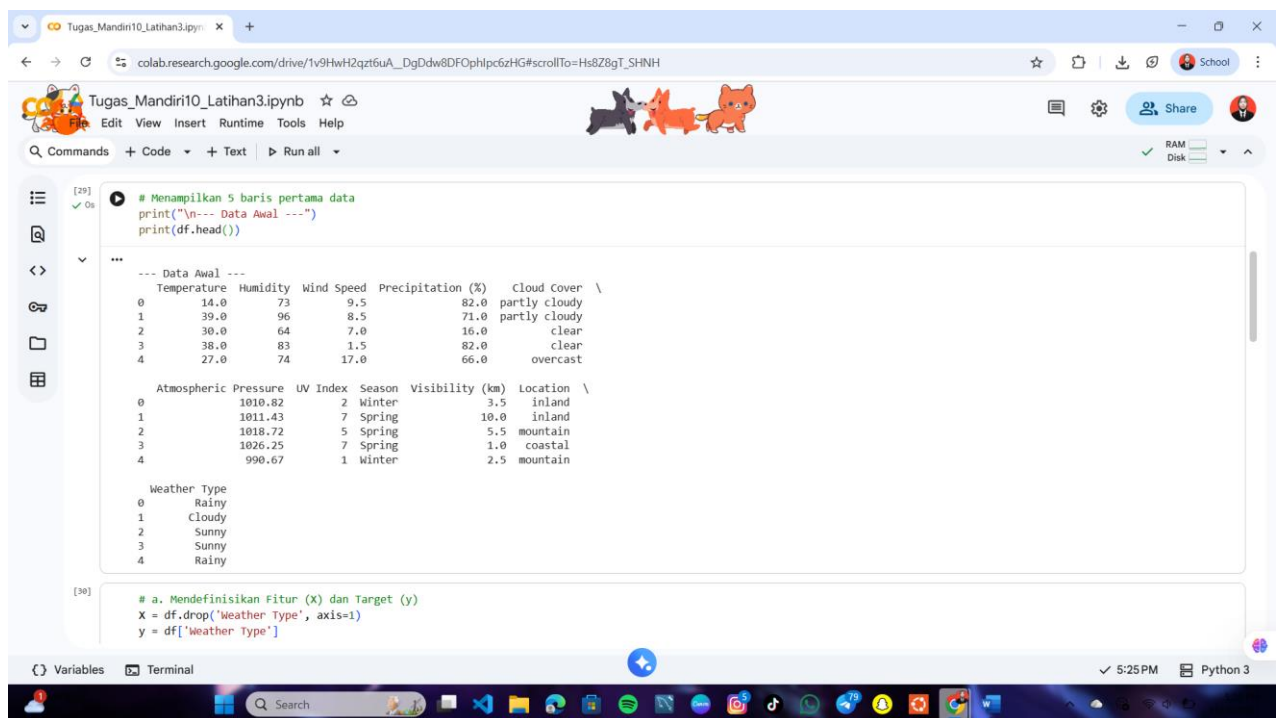
Mounted at /content/gdrive

[24] #Memanggil data set lewat gdrive
path = "/content/gdrive/MyDrive/Praktikum-Tugas_MachineLearning/Praktikum10"

[28] df = pd.read_csv('/content/gdrive/MyDrive/Praktikum-Tugas_MachineLearning/Praktikum10/Data/weather_classification_data.csv')

[29] # Menampilkan 5 baris pertama data
print("\n--- Data Awal ---")
print(df.head())

--- Data Awal ---
Temperature Humidity Wind Speed Precipitation (%) Cloud Cover \
0 14.0 73 9.5 82.0 partly cloudy
1 39.0 96 8.5 71.0 partly cloudy
2 30.0 64 7.0 16.0 clear
3 38.0 83 1.5 82.0 clear
4 27.0 74 17.0 66.0 overcast
```



The screenshot shows the same Google Colab notebook, but the output of the previous cell is visible. The data is displayed in a table format. The next cell defines the features (X) and the target variable (y) for the model.

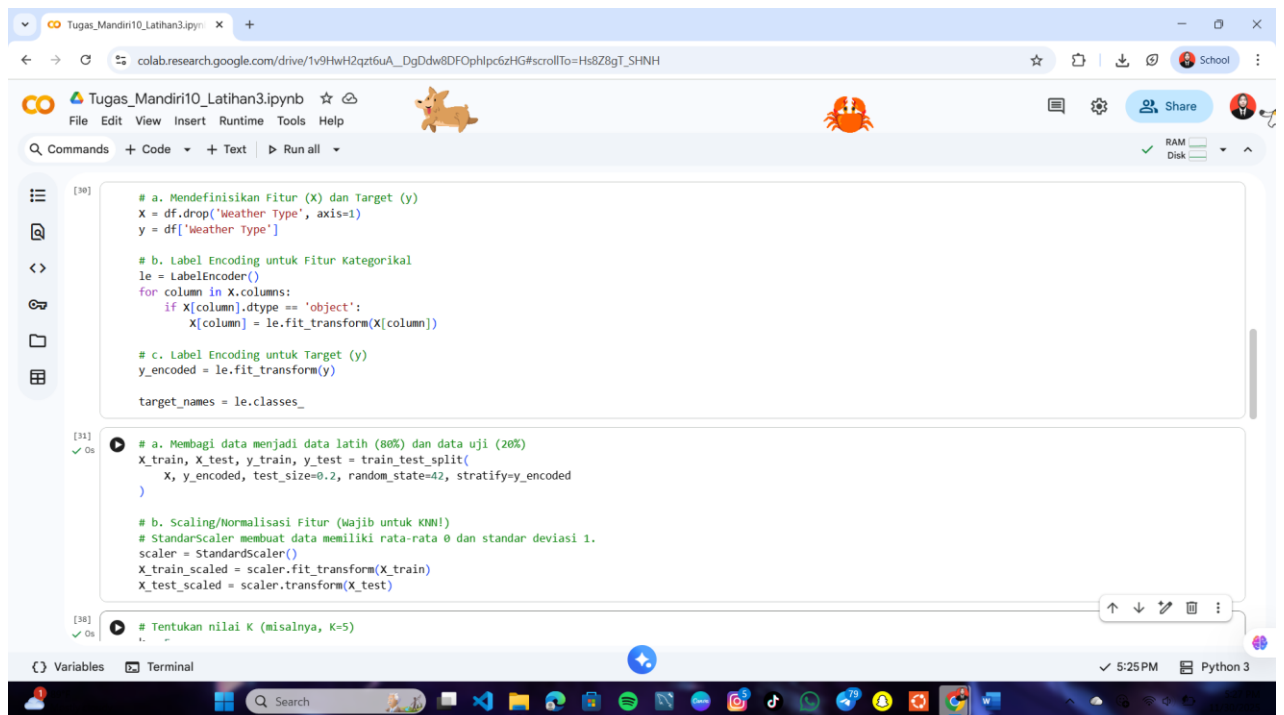
```
[29] # Menampilkan 5 baris pertama data
print("\n--- Data Awal ---")
print(df.head())

--- Data Awal ---
Temperature Humidity Wind Speed Precipitation (%) Cloud Cover \
0 14.0 73 9.5 82.0 partly cloudy
1 39.0 96 8.5 71.0 partly cloudy
2 30.0 64 7.0 16.0 clear
3 38.0 83 1.5 82.0 clear
4 27.0 74 17.0 66.0 overcast

Atmospheric Pressure UV Index Season Visibility (km) Location \
0 1010.82 2 Winter 3.5 inland
1 1011.43 7 Spring 10.0 inland
2 1018.72 5 Spring 5.5 mountain
3 1026.25 7 Spring 1.0 coastal
4 990.67 1 Winter 2.5 mountain

Weather Type
0 Rainy
1 Cloudy
2 Sunny
3 Sunny
4 Rainy

[30] # a. Mendefinisikan Fitur (X) dan Target (y)
X = df.drop('Weather Type', axis=1)
y = df['Weather Type']
```



Tugas_Mandiri10_Latihan3.ipynb

```
[30] # a. Mendefinisikan Fitur (X) dan Target (y)
X = df.drop('Weather Type', axis=1)
y = df['Weather Type']

# b. Label Encoding untuk Fitur Kategorikal
le = LabelEncoder()
for column in X.columns:
    if X[column].dtype == 'object':
        X[column] = le.fit_transform(X[column])

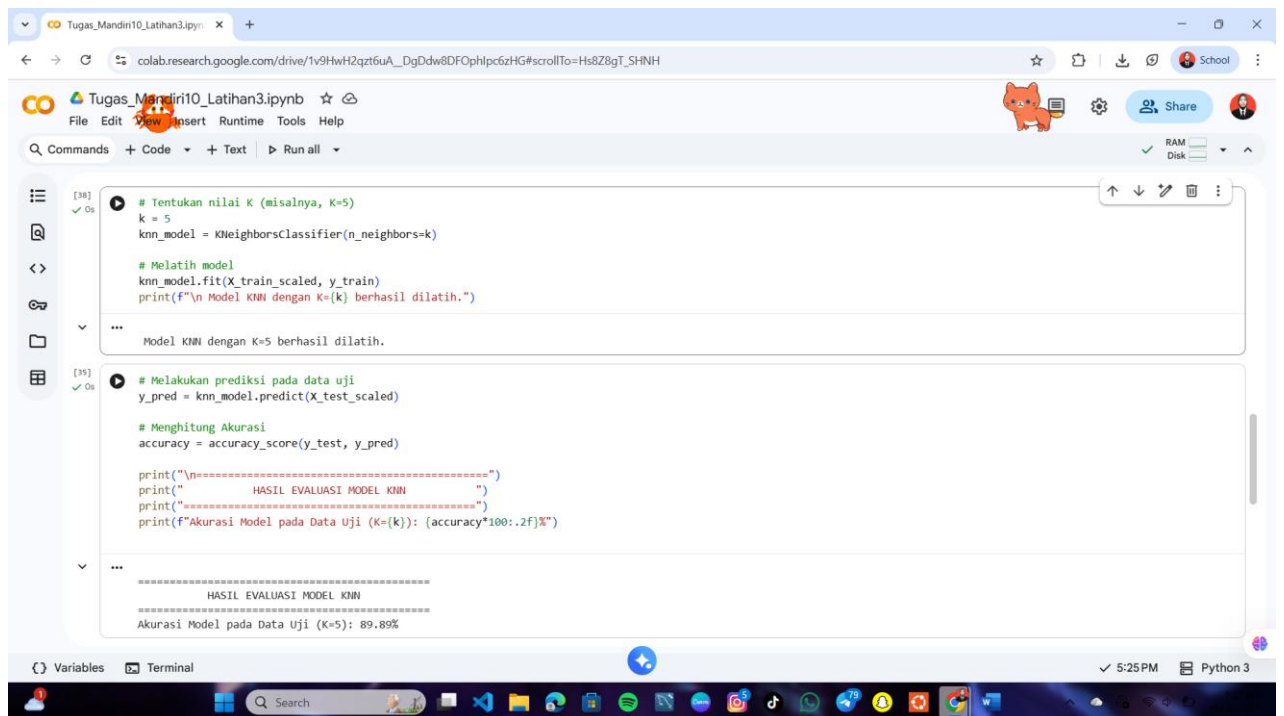
# c. Label Encoding untuk Target (y)
y_encoded = le.fit_transform(y)
target_names = le.classes_

[31] # a. Membagi data menjadi data latih (80%) dan data uji (20%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

# b. Scaling/Normalisasi Fitur (wajib untuk KNN!)
# StandardScaler membuat data memiliki rata-rata 0 dan standar deviasi 1.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

[38] # Tentukan nilai K (misalnya, K=5)
```

5:25 PM Python 3



Tugas_Mandiri10_Latihan3.ipynb

```
[38] # Tentukan nilai K (misalnya, K=5)
k = 5
knn_model = KNeighborsClassifier(n_neighbors=k)

# Melatih model
knn_model.fit(X_train_scaled, y_train)
print(f"\n Model KNN dengan K={k} berhasil dilatih.")

...
Model KNN dengan K=5 berhasil dilatih.

[39] # Melakukan prediksi pada data uji
y_pred = knn_model.predict(X_test_scaled)

# Menghitung Akurasi
accuracy = accuracy_score(y_test, y_pred)

print("\n=====")
print("          HASIL EVALUASI MODEL KNN          ")
print("=====")
print(f"Akurasi Model pada Data Uji (K={k}): {accuracy*100:.2f}%")

...
=====
HASIL EVALUASI MODEL KNN
=====
Akurasi Model pada Data Uji (K=5): 89.89%
```

5:25 PM Python 3


```
[35]: print("===== HASIL EVALUASI MODEL KNN =====")
print(f"Akurasi Model pada Data Uji (K={k}): {accuracy*100:.2f}%")

===== HASIL EVALUASI MODEL KNN =====
Akurasi Model pada Data Uji (K=5): 89.89%

[37]: from sklearn.metrics import classification_report

# Menampilkan Classification Report (Precision, Recall, F1-Score)
print("\n--- Classification Report ---")
print(classification_report(y_test, y_pred, target_names=target_names))

--- Classification Report ---
precision    recall  f1-score   support

 Cloudy      0.85      0.89      0.87         660
  Rainy      0.87      0.91      0.89         660
   Snowy     0.94      0.92      0.93         660
   Sunny     0.94      0.89      0.91         660

 accuracy    0.90      0.90      0.90        2640
 macro avg   0.90      0.90      0.90        2640
weighted avg   0.90      0.90      0.90        2640
```

1. Penjelasan Langkah-Langkah Analisis

- Pra-pemrosesan Data (Data Preprocessing):
 - Penyandian Label (Label Encoding): Fitur-fitur yang berupa teks atau data kategorikal (seperti Cloud Cover, Season, dan Location) diubah menjadi nilai numerik. Hal ini wajib dilakukan agar algoritma KNN dapat menghitung jarak.
 - Pembagian Data: Data dibagi menjadi 80% set pelatihan (training) dan 20% set pengujian (testing). Data pelatihan digunakan untuk "mengajari" model, dan data pengujian digunakan untuk mengukur seberapa baik kinerja model pada data baru.
 - Normalisasi Skala (Standard Scaling): Ini adalah langkah krusial untuk KNN. Semua nilai fitur disesuaikan (dinormalisasi) sehingga memiliki rata-rata nol dan standar deviasi satu. Hal ini mencegah fitur dengan rentang nilai besar (seperti Tekanan Atmosfer) mendominasi perhitungan jarak, sehingga setiap fitur memiliki bobot yang sama pentingnya dalam proses klasifikasi.
- Pelatihan Model: Model KNN diinisialisasi dengan menetapkan nilai K=5 (lima tetangga terdekat) dan dilatih menggunakan data pelatihan yang telah dinormalisasi.
- Evaluasi Model: Model yang sudah dilatih kemudian digunakan untuk memprediksi jenis cuaca pada data pengujian. Hasil prediksi ini dibandingkan dengan hasil sebenarnya untuk menghitung metrik performa.

2. Interpretasi Hasil Perhitungan dan Prediksi

- Akurasi Total: Model mencapai akurasi sebesar 89.89%. Ini berarti model mampu memprediksi jenis cuaca dengan benar hampir 90 kali dari 100 kali percobaan pada data yang belum pernah dilihat sebelumnya.
- Presisi (Precision): Model menunjukkan presisi tertinggi (0.94) untuk cuaca Sunny. Artinya, ketika model memprediksi hari itu akan Cerah (Sunny), prediksi tersebut benar 94% dari waktu. Presisi yang tinggi menunjukkan tingkat keandalan model.
- Recall (Recall): Model juga menunjukkan nilai Recall yang tinggi di atas 0.90 untuk sebagian besar kelas. Recall mengukur kemampuan model untuk menemukan semua

kejadian positif. Sebagai contoh, dari semua hari yang sebenarnya Hujan (Rainy), model berhasil mengidentifikasi 91% di antaranya.

- Keseimbangan (F1-Score): Nilai F1-Score (rata-rata harmonik dari Presisi dan Recall) untuk semua kategori cuaca berada di kisaran 0.89 hingga 0.93. Hal ini menunjukkan bahwa model tidak hanya akurat, tetapi juga memiliki keseimbangan yang baik antara Presisi dan Recall di seluruh jenis cuaca.

3. Kesimpulan Akhir

Model K-Nearest Neighbors yang dikembangkan untuk prediksi cuaca ini memiliki kinerja sangat baik dengan akurasi keseluruhan mendekati 90%. Keberhasilan model ini sangat ditunjang oleh langkah Standard Scaling yang efektif mengatasi perbedaan skala antar fitur.

LINK GITHUB UPLOAD TUGAS : <https://github.com/Yurida26/Machine-Learning/tree/e3c4c2171e25ad0b0dce96d63bd10e2c52e3ee34/Praktikum10>

